



הצעת פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון: 714918)

בנושא

שרת FTP לאחסון והורדת קבצים



שם הסטודנט: אוריאל טוטיאשווילי

ת"ז: 215750779

מנחה: אילן פרץ

תאריך: 13/1/2026

שם מכללה: כנפי רוח, קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

1.	תיאור הנושא.....	3
2.	רקע תיאורטי.....	4
3.	תיאור הפרויקט.....	6
4.	הגדרת הבעיה האלגוריתמית.....	8
5.	הליכים עיקריים בפתרון הבעיה בטכנולוגיות הנדסה מתקדמות.....	10
6.	הליכים עיקריים בתחום למידת מכונה- לא רלוונטי.....	14
7.	הליכים עיקריים מע' הפעלה/רשתות/תקשורת/אבטחה - לא רלוונטי.....	14
8.	תיאור פרוטוקולי תקשורת.....	14
9.	פיתוחים עתידיים.....	16
10.	תיאור טכנולוגיה הנדסית.....	16
11.	מסד נתונים.....	17
12.	פרטים פורמליים.....	19

1. תיאור הנושא

התחום שבו עוסק פרויקט זה הוא **תחום העברת ואחסון קבצים ברשתות מחשבים**, כחלק מעולמות **מערכות תקשורת, רשתות מחשבים וניהול מידע דיגיטלי**. תחום זה מהווה נדבך מרכזי בתשתיות המידע המודרניות, ומשמש בסיס לפעילות של מערכות ארגוניות, שירותי ענן, מערכות גיבוי, ושיתוף מידע בין משתמשים ומערכות קצה.

המטרה המרכזית של תחום העברת ואחסון קבצים היא **לאפשר העברה יעילה, אמינה וחסכונית של מידע דיגיטלי בין גורמים שונים**, תוך שמירה על שלמות הנתונים, זמני תגובה סבירים וניצול מיטבי של משאבי הרשת והמערכת. התחום שואף להתמודד עם נפחים הולכים וגדלים של מידע, מספר רב של משתמשים, ומגבלות פיזיות ולוגיות של תשתיות תקשורת.

תחום זה מתאפיין במספר מאפיינים מרכזיים:

- עבודה עם **קבצים בנפחים משתנים**, החל מקבצים קטנים ועד קבצים גדולים מאוד.
 - שימוש בפרוטוקולי תקשורת להעברת מידע מעל רשתות מקומיות ואינטרנט.
 - תלות גבוהה ב**רוחב פס**, זמני השהיה (Latency) ויציבות החיבור.
 - צורך בניהול יעיל של משאבי מערכת כגון זיכרון, מעבד ואחסון.
 - חשיבות לשמירה על **שלמות הנתונים** במהלך ההעברה והאחסון.
- לצד חשיבותו, התחום מתמודד עם אתגרים משמעותיים:
- **צריכת רוחב פס גבוהה** בעת העברת קבצים גדולים, העלולה לגרום לעומס על הרשת ולהאטת שירותים אחרים.
 - **זמני העברה ארוכים**, במיוחד בסביבות עם חיבורים איטיים או לא יציבים.
 - **התמודדות עם שגיאות תקשורת**, ניתוקים ואיבוד נתונים במהלך ההעברה.
 - הצורך בייעול תעבורת המידע ללא פגיעה בשלמות הקובץ.
 - איזון בין יעילות העברה לבין מורכבות חישובית של תהליכי עיבוד הנתונים.
- אתגרים אלו יוצרים צורך בפתרונות מתקדמים המשלבים שיטות לניהול תעבורה, חלוקת מידע והקטנת נפחי נתונים, על מנת לשפר את ביצועי מערכות העברת ואחסון הקבצים.

2. רקע תיאורי

בתחום העברת ואחסון קבצים ברשתות מחשבים קיימת בעיה מרכזית הנוגעת לנפח הנתונים המועברים והמאוחסנים. קבצים דיגיטליים, ובפרט קבצים גדולים כגון סרטונים, מסמכים מורכבים וארכיונים, דורשים משאבי רשת ואחסון משמעותיים. העברת קבצים אלו כפי שהם, ללא עיבוד מוקדם, מובילה להעמסת רוחב הפס, להארכת זמני העברה ולניצול לא יעיל של משאבי המערכת.

הבעיה האלגוריתמית המוגדרת בפרויקט היא הפחתת נפח המידע של קבצים דיגיטליים לצורך העברה ואחסון ברשת, תוך שמירה מלאה על תוכן הקובץ המקורי ואפשרות לשחזור מדויק שלו. באופן פורמלי, מדובר בבעיה מסוג דחיסת נתונים חסרת אובדן (Lossless Data Compression). שבה האלגוריתם מקבל קלט של רצף נתונים ומחזיר ייצוג דחוס יותר, כך שקיים תהליך הפוך המאפשר שחזור מלא של הנתונים ללא כל אובדן מידע.

בעיה זו היא בעיה אלגוריתמית מובהקת, שכן היא עוסקת בזיהוי תבניות וחזרות בנתונים, בניהול מבני נתונים מתאימים ובביצוע טרנספורמציה של הקלט לפלט יעיל יותר מבחינת נפח. הגדרת הבעיה האלגוריתמית נובעת ישירות מהאתגרים המרכזיים שתוארו בסעיף תיאור הנושא:

- הצורך ביעול העברת קבצים ברשתות מחשבים
- מגבלות רוחב פס וזמני העברה ארוכים
- דרישה לשימוש חסכוני במשאבי אחסון

באמצעות הפחתת נפח הנתונים המועברים והמאוחסנים, דחיסת נתונים חסרת אובדן מספקת מענה ישיר לאתגרים אלו. האלגוריתם מאפשר הקטנת כמות המידע הזורם ברשת ובכך תורם לשיפור יעילות ההעברה ולשימוש מיטבי במשאבי המערכת, תוך שמירה על שלמות הקבצים.

דחיסת נתונים – היבט אלגוריתמי

העברת קבצים גדולים ברשת יוצרת עומס על רוחב הפס ודורשת זמן רב. לפיכך נעשה שימוש בדחיסת נתונים, המתמקדת בצמצום נפח המידע תוך שמירה על תוכנו המקורי. בפרויקט נעשה שימוש בדחיסה ללא אובדן מידע, שבה הקובץ המשוחזר זהה לחלוטין למקור. קיימים מספר אלגוריתמים מרכזיים בתחום זה.

אלגוריתמים מרכזיים לדחיסה ללא אובדן

Huffman Coding הוא אלגוריתם דחיסה ללא אובדן מידע המבוסס על ניתוח תדירות הופעת התווים בקלט. האלגוריתם בונה עץ בינארי שבו תווים בעלי תדירות גבוהה מקבלים קוד בינארי קצר יותר, בעוד שתווים נדירים מקבלים קוד ארוך יותר. שיטה זו מבטיחה קידוד אופטימלי מבחינת אורך ממוצע של הקוד, אך אינה מנצלת חזרות של רצפי תווים אלא פועלת ברמת התו הבודד.

LZ77 הוא אלגוריתם דחיסה המבוסס על זיהוי חזרות מקומיות בזרם הנתונים. האלגוריתם עושה שימוש בחלון הזזה (Sliding Window) ומחליף רצפים חוזרים בהפניות לאזורים קודמים בקלט, המיוצגות על ידי מרחק ואורך. שיטה זו מתאימה במיוחד לזרמי נתונים רציפים, אך יעילותה תלויה בגודל החלון ובמרחק בין החזרות.

LZ78 פועל על ידי בניית מילון של רצפים חדשים בזמן קריאת הקלט. בכל פעם שמופיע רצף שאינו קיים במילון, הוא מתווסף אליו ומיוצג באמצעות אינדקס למילון ותו נוסף. גישה זו מאפשרת ייצוג קומפקטי של חזרות ארוכות ומגוונות, תוך שמירה על תהליך דחיסה ופענוח פשוטים וברורים.

LZW הוא שיפור של LZ78, שבו אין צורך לשלוח את התו הנוסף בכל שלב, אלא רק אינדקסים למילון המתפתח. שיטה זו מיעלת את תהליך הקידוד ומקטינה את נפח הפלט, במיוחד בקבצים טקסטואליים. עם זאת, האלגוריתם מורכב יותר ליישום ולניתוח תאורטי בהשוואה ל-LZ78.

שם האלגוריתם	יתרונות	חסרונות	זמן ריצה
Huffman code	קידוד אופטימלי לפי תדירות, פשוט מבחינה מתמטית, מבטיח קידוד מינימלי	אינו מזהה חזרות, דורש מעבר מלא על הנתונים לבניית עץ, פחות יעיל לבד	$O(n \log n)$
LZ77	אינו דורש מילון קבוע מראש, יעיל בזיהוי חזרות סמוכות, מתאים לזרימת נתונים רציפה	פחות יעיל כאשר החזרות מרוחקות, צורך חלון חיפוש, ביצועים תלויים בגודל החלון	$O(n)$
LZ78	בניית מילון דינמית, מתאים למגוון רחב של קבצים, פשוט להבנה ולשחזור, דחיסה יציבה	צורך אחסון מילון, יעילות נמוכה יחסית בקבצים קצרים מאוד	$O(n)$
LZW	אינו דורש שליחת מילון בנפרד, יעיל מאוד לקבצים טקסטואליים, נפוץ בתעשייה	מורכב יותר מ-LZ78, בעיות רישוי היסטוריות, פחות שקוף להסבר תאורטי	$O(n)$

3. תיאור הפרויקט

המערכת היא אתר אינטרנט לאחסון ולהעברת קבצים, המיועד למשתמשים פרטיים הזקוקים לפתרון פשוט ונגיש לשמירת קבצים ולהעברתם בין מכשירים שונים. המערכת מאפשרת למשתמש לשמור קבצים על שרת מרוחק, ולגשת אליהם מכל מקום ובכל זמן באמצעות דפדפן אינטרנט, ללא צורך בהתקנת תוכנה או שימוש באמצעי אחסון פיזיים. השימוש במערכת מתבצע בצורה אינטואיטיבית: המשתמש נרשם לאתר, מתחבר לחשבון האישי שלו, מעלה קבצים מהמחשב האישי, וצופה ברשימת הקבצים השמורים עבורו. בכל שלב המשתמש יכול להוריד קבצים חזרה למחשב או למחוק קבצים שאינם נחוצים עוד.

רכיבי המערכת מנקודת מבט המשתמש

הרשמה והתחברות- המערכת כוללת ממשק הרשמה המאפשר יצירת חשבון אישי, וכן מסך התחברות באמצעות פרטי הזדהות. לאחר ההתחברות, המשתמש מקבל גישה לאזור האישי שלו ולכל הקבצים השייכים לו בלבד.

אזור אישי ורשימת קבצים- באזור האישי מוצגת למשתמש רשימת הקבצים שלו. הרשימה מאפשרת לראות בצורה ברורה אילו קבצים שמורים במערכת, ומסייעת בניהול מסודר של המידע.

העלאת קבצים- המערכת מאפשרת למשתמש לבחור קובץ מהמחשב האישי ולהעלות אותו לאחסון באתר באמצעות לחיצה פשוטה. תהליך זה מיועד להיות ברור ונוח גם למשתמשים ללא ידע טכני.

פעולות על קבצים- לצד כל קובץ קיימות פעולות בסיסיות:

- **הורדת קובץ** – העברת הקובץ מהמערכת חזרה למחשב המשתמש

- **מחיקת קובץ** – הסרת הקובץ מהאחסון במערכת

פעולות אלו מעניקות למשתמש שליטה מלאה על הקבצים השמורים עבורו.

המערכת מספקת מענה ישיר לאתגרים המרכזיים בתחום העברת ואחסון קבצים, כפי שתוארו בסעיף

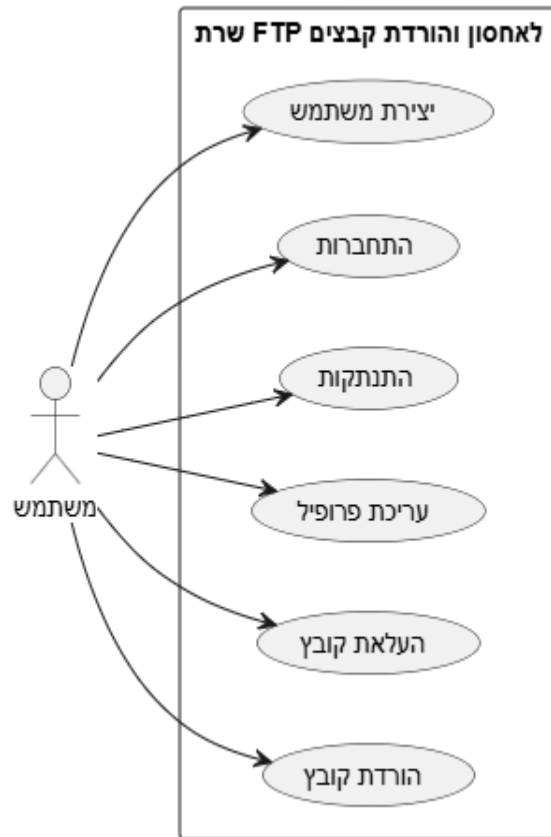
תיאור הנושא:

- מאפשרת שמירה והעברה של קבצים בנפחים גדולים, ללא תלות במגבלות של אמצעי העברה בסיסיים.

- מספקת נגישות מלאה למידע, מכל מחשב המחובר לרשת.

- יוצרת סדר וריכוז של קבצים, במקום אחד ברור ונוח לשימוש.

להלן **תרשימים תרחישים Use Case** של התהליכים המרכזיים במערכת התוכנה:



1. יצירת משתמש

מקרה שימוש זה מתאר את תהליך ההרשמה למערכת.

המשתמש מזין פרטים בסיסיים הנדרשים לפתיחת חשבון, כגון שם משתמש וסיסמה. לאחר הזנת הפרטים, המערכת בודקת שהנתונים תקינים וששם המשתמש אינו קיים כבר במערכת.

במידה והבדיקות עוברות בהצלחה, נוצר חשבון חדש והמשתמש יכול להתחבר למערכת.

2. התחברות

במקרה שימוש זה המשתמש נכנס למערכת באמצעות פרטי הזדהות אישיים. המשתמש מזין שם משתמש וסיסמה, והמערכת בודקת את התאמתם לנתונים השמורים.

אם הפרטים נכונים, המשתמש מקבל גישה לאזור האישי שלו ויכול לבצע פעולות כמו העלאה והורדה של קבצים.

3. התנתקות

מקרה שימוש זה מתאר את סיום העבודה של המשתמש במערכת. המשתמש בוחר להתנתק, והמערכת סוגרת את החיבור שלו בצורה מסודרת. פעולה זו נועדה לשמור על אבטחת המידע ולמנוע גישה לא מורשית לחשבון לאחר סיום השימוש.

4. עריכת פרופיל

במקרה שימוש זה המשתמש יכול לעדכן את פרטיו האישיים במערכת. המשתמש בוחר אילו פרטים לשנות, כגון סיסמה או פרטים מזהים אחרים. המערכת שומרת את השינויים ומוודאת שהמידע החדש מעודכן ונשמר בצורה תקינה.

5. העלאת קובץ

מקרה שימוש זה מתאר את תהליך שמירת הקבצים של המשתמש בשרת. המשתמש בוחר קובץ מהמחשב האישי ומבקש להעלות אותו למערכת. המערכת בודקת שלמשתמש יש הרשאה לבצע את הפעולה, ולאחר מכן הקובץ עובר תהליך דחיסה לצמצום נפחו. לאחר הדחיסה הקובץ נשמר בשרת, והמשתמש מקבל הודעה על הצלחת הפעולה.

6. הורדת קובץ

מקרה שימוש זה מתאר את תהליך קבלת קובץ מהמערכת אל מחשב המשתמש. המשתמש בוחר קובץ מתוך רשימת הקבצים השמורים ומבקש להוריד אותו. המערכת בודקת הרשאות, שולפת את הקובץ מהאחסון ומבצעת פריסה (שחזור) של הקובץ למצבו המקורי. בסיום התהליך הקובץ מועבר למשתמש בצורה תקינה ומלאה.

4. הגדרת הבעיה האלגוריתמית

בתחום העברת ואחסון קבצים ברשתות מחשבים קיימת בעיה מרכזית הנוגעת לנפח הנתונים המועברים והמאוחסנים. קבצים דיגיטליים, ובפרט קבצים גדולים כגון סרטונים, מסמכים מורכבים וארכיונים, דורשים משאבי רשת ואחסון משמעותיים. העברת קבצים אלו כפי שהם, ללא עיבוד מוקדם, מובילה להעמסת רוחב הפס, להארכת זמני העברה ולניצול לא יעיל של משאבי המערכת. הבעיה האלגוריתמית המוגדרת בפרויקט היא הפחתת נפח המידע של קבצים דיגיטליים לצורך העברה ואחסון ברשת, תוך שמירה מלאה על תוכן הקובץ המקורי ואפשרות לשחזור מדויק שלו. באופן פורמלי, מדובר בבעיה מסוג דחיסת נתונים חסרת אובדן, (Lossless Data Compression) שבה האלגוריתם מקבל קלט של רצף נתונים ומחזיר ייצוג דחוס יותר, כך שקיים תהליך הפוך המאפשר שחזור מלא של הנתונים ללא כל אובדן מידע. בעיה זו היא בעיה אלגוריתמית מובהקת, שכן היא עוסקת בזיהוי תבניות וחזרות בנתונים, בניהול מבני נתונים מתאימים ובביצוע טרנספורמציה של הקלט לפלט יעיל יותר מבחינת נפח. הגדרת הבעיה האלגוריתמית נובעת ישירות מהאתגרים המרכזיים שתוארו בסעיף תיאור הנושא:

- הצורך בייעול העברת קבצים ברשתות מחשבים
- מגבלות רוחב פס וזמני העברה ארוכים
- דרישה לשימוש חסכוני במשאבי אחסון

באמצעות הפחתת נפח הנתונים המועברים והמאוחסנים, דחיסת נתונים חסרת אובדן מספקת מענה ישיר לאתגרים אלו. האלגוריתם מאפשר הקטנת כמות המידע הזורם ברשת ובכך תורם לשיפור יעילות ההעברה ושימוש מיטבי במשאבי המערכת, תוך שמירה על שלמות הקבצים.

האלגוריתם הנבחר:

האלגוריתם **LZ78** נבחר כאלגוריתם הדחיסה המרכזי במערכת, מאחר שהוא מספק מענה ישיר לדרישות הבעיה האלגוריתמית: צמצום נפח נתונים לצורך העברה ואחסון, תוך שמירה מלאה על שלמות המידע. האלגוריתם מבצע דחיסה ללא אובדן, ולכן מאפשר שחזור מדויק של הקובץ המקורי לאחר ההעברה.

LZ78 פועל על ידי קריאת הקלט משמאל לימין ובניית מילון דינמי של רצפים חוזרים המופיעים בנתונים. בכל שלב, האלגוריתם מזהה את הרצף הארוך ביותר שכבר קיים במילון, ומוסיף רצף חדש המורכב ממנו ומתו נוסף. כל רצף מיוצג באמצעות זוג הכולל אינדקס למילון ותו, ובכך מתקבל ייצוג קומפקטי יותר של הנתונים.

גישה זו מאפשרת ניצול יעיל של חזרות במידע, גם כאשר מבנה הקובץ אינו ידוע מראש, ומאפשרת עיבוד הדרגתי של הנתונים – תכונה חשובה בעבודה עם קבצים גדולים ובהעברה ברשת.

אופן פעולת האלגוריתם

האלגוריתם פועל לפי השלבים הבאים:

1. אתחול מילון ריק, שבו האינדקס 0 מייצג רצף ריק.
 2. קריאת הקלט מהתו הראשון ועד סופו.
 3. בכל שלב, האלגוריתם מחפש את הרצף הארוך ביותר מהקלט הנוכחי שכבר קיים במילון.
 4. לאחר זיהוי הרצף, האלגוריתם קורא את התו הבא בקלט שעדיין לא נכלל ברצף.
 5. נוצר פלט בצורת זוג:
 - אינדקס הרצף הקיים במילון
 - התו החדש שנוסף
 6. הרצף החדש (הרצף הקיים + התו החדש) מתווסף למילון עם אינדקס חדש.
 7. האלגוריתם ממשיך לקרוא את הקלט מנקודת הסיום של הרצף שנמצא.
 8. התהליך נמשך עד לסיום הקלט.
- באופן זה, במקום לשלוח או לאחסן כל תו בנפרד, האלגוריתם מייצג רצפים שלמים באמצעות זוגות קצרים, ובכך מצמצם את נפח הנתונים הכולל.

הפעולה של האלגוריתם ניתנת לתיאור מילולי באופן הבא:

התחל עם מילון ריק.

קרא את הקלט משמאל לימין.

כל עוד נותרו תווים בקלט:

מצא את הרצף הארוך ביותר שכבר מופיע במילון ומתחיל במיקום הנוכחי בקלט.

קרא את התו הבא שאינו חלק מהרצף שנמצא.

פלט זוג המורכב מהאינדקס של הרצף במילון ומהתו החדש.

הוסף למילון רצף חדש המורכב מהרצף שנמצא ומהתו החדש.

המשך לקרוא את הקלט מהמיקום הבא.

סיים כאשר כל הקלט עובד.

תיאור זה מדגיש את אופי העבודה ההדרגתית של האלגוריתם ואת בניית המילון תוך כדי תהליך הדחיסה.

דוגמה להמחשה:

עבור המחרוזת: ABABABA

האלגוריתם יבנה בהדרגה מילון של רצפים כגון:

- A •
- B •
- AB •
- ABA •

במקום לייצג כל תו בנפרד, הרצפים החוזרים מוחלפים בייצוג קצר המבוסס על אינדקס מהמילון ותו נוסף, וכך נפח המידע הכולל קטן.

5. הליכים עיקריים בפתרון הבעיה בטכנולוגיות הנדסה מתקדמות

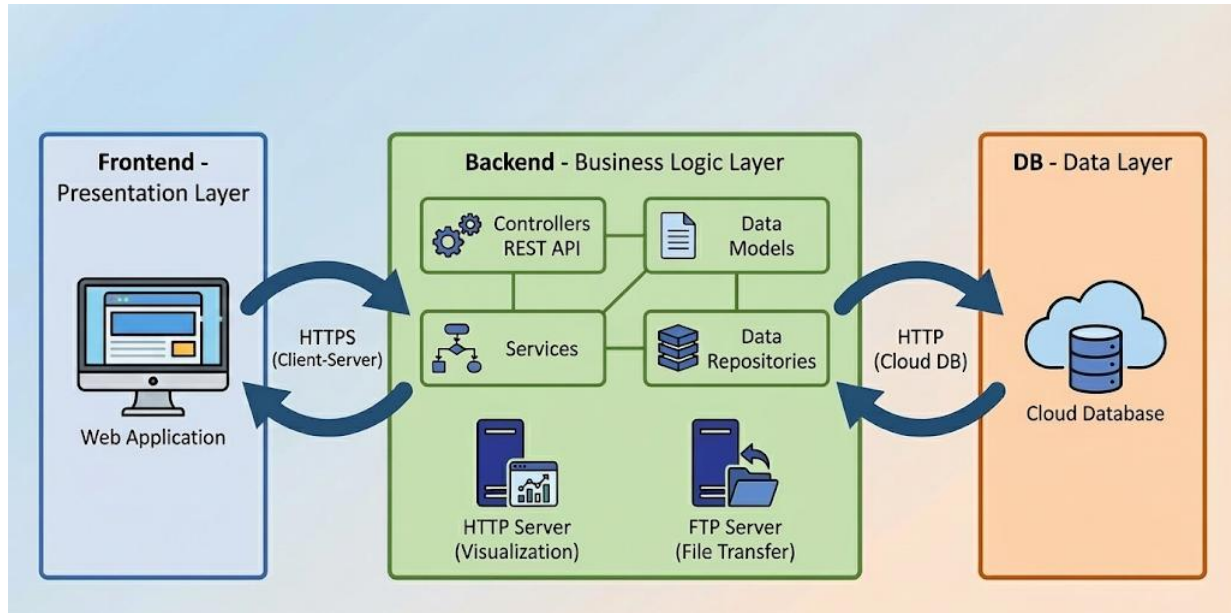
בסעיף זה מתואר המבנה הטכני של המערכת והאופן שבו רכיביה פועלים יחד לצורך מימוש תהליכי העברת ואחסון קבצים. התיאור נעשה מנקודת מבט הנדסית, ומתמקד בארכיטקטורת המערכת, בזרימת המידע בין רכיביה, בשילוב בין פרוטוקולי התקשורת ובאופן יישום הפתרון האלגוריתמי שנבחר. סעיף זה אינו עוסק בחוויית משתמש או בממשק גרפי, אלא בתכנון ובפעולה הפנימית של המערכת.

ארכיטקטורת שלוש השכבות (Three-Tier Architecture)

המערכת מבוססת על ארכיטקטורת שלוש שכבות, המאפשרת הפרדה ברורה של תחומי אחריות בין רכיבי המערכת. ארכיטקטורה זו נפוצה במערכות שרת-לקוח ומספקת סדר הנדסי, תחזוקה נוחה והפרדה בין שכבות העיבוד השונות.

ארכיטקטורת שלוש השכבות כוללת:

1. שכבת תצוגה (Presentation Layer)
2. שכבת לוגיקה יישומית (Application / Business Logic Layer)
3. שכבת נתונים (Data Layer)



חלוקה זו מאפשרת לכל שכבה לפעול בתחום אחריות מוגדר, תוך תקשורת מבוקרת עם שאר השכבות. להלן הסבר על התרשים:

1. שכבת התצוגה (Presentation Layer)

שכבת התצוגה אחראית על יצירת נקודת הגישה למערכת ועל שליחת בקשות לפעולות כגון התחברות, העלאת קבצים והורדתם. שכבה זו ממומשת באמצעות ממשק Web ופועלת מעל פרוטוקול HTTP. מבחינה טכנית, שכבה זו משמשת כמתווך בלבד ואינה מבצעת עיבוד נתונים, דחיסה או ניהול אחסון, אלא שולחת בקשות לשכבת הלוגיקה היישומית ומציגה את התשובות המתקבלות ממנה.

2. שכבת הלוגיקה היישומית (Application / Business Logic Layer)

שכבת הלוגיקה היישומית מהווה את הליבה ההנדסית של המערכת. שכבה זו אחראית על ניהול תהליכי העבודה, קבלת החלטות וביצוע הפעולות המרכזיות של המערכת. במסגרת זו מתבצע התיאום בין שכבת התצוגה, שרת ה-FTP ושכבת הנתונים.

בשכבה זו מתבצעים:

- ניהול חיבורי משתמשים ואימות זהות
- בדיקת הרשאות וגישה לקבצים
- ניהול תהליכי העלאת והורדה
- הפעלת תהליך דחיסת קבצים לפני אחסון

- הפעלת תהליך פריסה (Decompression) לפני הורדה

מנגנון הדחיסה משולב כחלק בלתי נפרד משכבת הלוגיקה היישומית, ופועל כתהליך פנימי התומך בפעולת שרת ה-FTP ומאפשר ייעול של נפח הנתונים המועברים והמאוחסנים.

3. שכבת הנתונים (Data Layer)

שכבת הנתונים אחראית על אחסון הקבצים והמידע הנלווה אליהם. בפרויקט זה, שכבה זו כוללת את מאגר הקבצים המאוחסנים בשרת, כאשר הקבצים נשמרים לאחר שעברו תהליך דחיסה. בנוסף, נשמר מידע הדרוש לניהול הקבצים ולשחזורם, כגון מבנה תיקיות ונתונים הנדרשים לתהליך הפריסה. שכבה זו אינה עוסקת בלוגיקה יישומית או בדחיסה, אלא מתמקדת באחסון אמין, מסודר ויעיל של המידע.

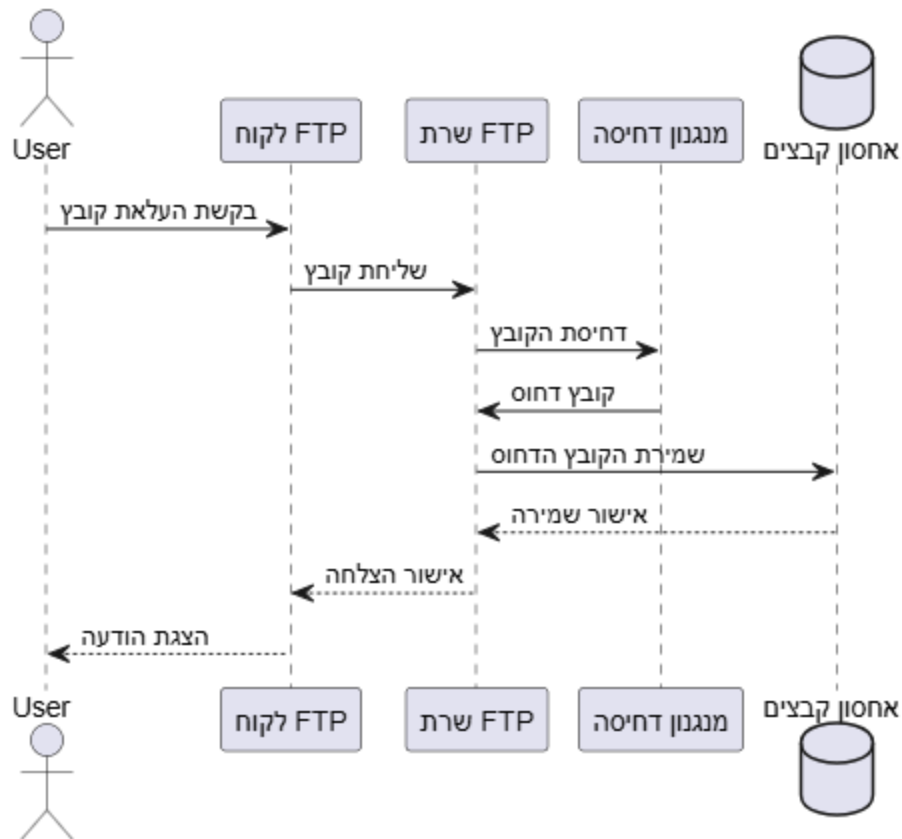
מודל צד לקוח / שרת (Client-Server)

המערכת פועלת לפי מודל צד לקוח/שרת, שבו צד הלקוח אחראי לשליחת בקשות בלבד, בעוד צד השרת אחראי על עיבוד הבקשות, ניהול הקבצים, הדחיסה והאחסון. חלוקה זו מבטיחה שהלוגיקה הקריטית והפעולות הרגישות מבוצעות בצד השרת בלבד, תוך שמירה על אבטחת המידע ושלמותו.

שילוב HTTP ל-FTP במערכת

המערכת משלבת בין שני פרוטוקולי תקשורת, כאשר לכל אחד תפקיד מוגדר וברור. שרת ה-HTTP המשמש כשכבת תיווך בין המשתמש לבין המערכת, ואחראי על קבלת בקשות והצגת מידע, בעוד ששרת ה-FTP אחראי על ניהול הקבצים והאכיפה בפועל של ההרשאות. בעת פעולה של העלאה או הורדה, הבקשה מועברת משרת ה-HTTP אל שרת ה-FTP. שרת ה-FTP מבצע אימות משתמש, בודק הרשאות גישה לקובץ המבוקש, ומחליט האם לאפשר את הפעולה. רק לאחר אישור הרשאות מתבצעת העברת הקובץ או שמירתו. גישה זו מבטיחה שהאבטחה והשליטה על הקבצים מתבצעות בשכבה המתאימה, ושהלוגיקה הקריטית הקשורה להרשאות אינה נמצאת בשכבת התצוגה או התיווך.

להלן תרשים רצף (Sequence Diagram) של תהליך העלאת קובץ



תרשים הרצף מתאר את רצף הפעולות בתהליך העלאת קובץ למערכת, ואת האינטראקציה בין הרכיבים השונים: משתמש, לקוח, שרת, FTP, מנגנון הדחיסה ושכבת האחסון.

הסבר לתרשים הרצף:

המשתמש יוזם בקשת העלאה, הקובץ מועבר לשרת ה-FTP, עובר תהליך דחיסה, נשמר באחסון ומתקבל אישור הצלחה המוחזר חזרה למשתמש. התרשים מדגים את מיקומו של מנגנון הדחיסה כחלק מרכזי מזרימת המידע.

זרימת המידע במערכת

זרימת המידע במערכת מתחילה בבקשת לקוח, ממשיכה לעיבוד לוגי בצד השרת, כוללת העברת קבצים באמצעות FTP ותהליך דחיסה אלגוריתמי, ומסתיימת באחסון הקובץ או בהעברתו חזרה ללקוח לאחר פריסה.

6. הליכים עיקריים בתחום למידת מכונה- לא רלוונטי

הצעה זו אינה עוסקת בלמידת מכונה, לכן סעיף זה לא רלוונטי.

7. הליכים עיקריים מע' הפעלה/רשתות/תקשורת/אבטחה - לא רלוונטי

הצעה זו אינה עוסקת מע' הפעלה/רשתות/תקשורת/אבטחת מידע, לכן סעיף זה לא רלוונטי.
הערה חשובה: התייחסות למודל שרת לקוח (Client/Server) מופיעה בהרחבה בסעיף 5 בהצעה.

8. תיאור פרוטוקולי תקשורת

פרוטוקולי תקשורת מהווים את הבסיס להעברת מידע ברשתות מחשבים. כל פרוטוקול מגדיר כללים ברורים לאופן שבו נתונים נשלחים, מתקבלים ומטופלים בין רכיבים שונים במערכת. בפרויקטים של העברת ואחסון קבצים נעשה שימוש במספר פרוטוקולים, כאשר כל אחד מהם מותאם לסוג פעולה שונה.

• HTTP / HTTPS (HyperText Transfer Protocol)

HTTP הוא פרוטוקול תקשורת נפוץ הפועל במודל של בקשה- תגובה, שבו הלקוח שולח בקשה לשרת והשרת מחזיר תגובה מתאימה. הפרוטוקול משמש בעיקר לתקשורת מבוססת דפדפן ולהעברת מידע לוגי כגון טקסט, נתונים ופעולות ניהול. הגרסה המאובטחת, HTTPS, עושה שימוש בהצפנה על מנת להגן על המידע המועבר בין הלקוח לשרת ולמנוע יירוט או שינוי של הנתונים במהלך התקשורת. בפרויקט זה HTTP/HTTPS משמש כשכבת הגישה למערכת, דרכה המשתמש מבצע פעולות ניהול כגון התחברות, צפייה ברשימת קבצים והפעלת בקשות לוגיות, מבלי לעסוק ישירות בהעברת הקבצים עצמם.

• FTP (File Transfer Protocol)

FTP הוא פרוטוקול ייעודי להעברת קבצים ברשת, המאפשר פעולות כגון העלאה, הורדה, מחיקה וניהול תיקיות על גבי שרת מרוחק. הפרוטוקול פועל באמצעות שני ערוצי תקשורת נפרדים: ערוץ פיקוד (Control Channel) להעברת פקודות וערוץ נתונים (Data Channel) להעברת הקבצים בפועל. FTP תומך במספר מצבי עבודה, כגון Active ו-Passive, וכן במצבי העברת נתונים שונים, כאשר מטרתו העיקרית היא העברה אמינה ומבוקרת של קבצים.

בפרויקט זה FTP משמש כפרוטוקול הליבה להעברת הקבצים, והוא אחראי על ניהול האחסון, אכיפת הרשאות והעברת קבצים בין השרת ללקוח בצורה יעילה ומדויקת.

• **TCP/IP (Transmission Control Protocol / Internet Protocol)**

TCP/IP מהווה את שכבת התשתית של התקשורת ברשת. TCP אחראי על תקשורת אמינה, בקרת זרימה, שמירה על סדר החבילות ושליחה חוזרת במקרה של אובדן נתונים IP. אחראי על ניתוב המידע בין נקודות שונות ברשת. שילוב זה מאפשר העברת מידע יציבה ושלמה גם במצבים של עומס רשת או העברת קבצים גדולים. בפרויקט זה TCP/IP משמש כתשתית שעליה פועלים פרוטוקולי HTTP ו-FTP, ומבטיח העברת נתונים אמינה בין רכיבי המערכת.

REST API (Representational State Transfer)

REST הוא סגנון ארכיטקטוני לתכנון ממשקי תקשורת בין מערכות. REST API מבוסס על שליחת בקשות HTTP לפעולות לוגיות, כאשר כל פעולה מיוצגת על ידי נקודת קצה (Endpoint).

ה-API משמש להעברת מידע בלבד, ללא תלות בממשק גרפי או במבנה דפי Web. נקודות קצה אופייניות מאפשרות פעולות כגון:

אימות משתמשים

קבלת מידע על משאבים

הפעלת תהליכים לוגיים

גישה זו מאפשרת תקשורת ברורה, מודולרית וסטנדרטית בין רכיבי מערכת שונים.

• **פורמט התקשורת- JSON**

JSON (JavaScript Object Notation) הוא פורמט טקסטואלי קל משקל לייצוג נתונים. הפורמט מבוסס על מבנה של זוגות מפתח-ערך ומאפשר ייצוג נוח של מידע מובנה כגון רשימות, אובייקטים ופרמטרים. JSON נפוץ מאוד בשימוש עם REST API בשל קריאותו, פשטותו והתאמתו להעברת מידע בין לקוח לשרת בצורה יעילה. בפרויקט זה JSON משמש כפורמט להעברת מידע ניהולי ונתונים לוגיים בין רכיבי המערכת.

9. פיתוחים עתידיים

1. שיפור חוויית המשתמש בממשק הווי-פיתוח ממשק נוח יותר, כולל גרירה ושחרור ותצוגת קבצים מקדימה(Preview).
2. סינון תוכן וקבצים- מנגנון למניעת העלאה של קבצים מזיקים או לא מורשים, לשיפור האבטחה.
3. אימות דו שלבי(Two Factor Authentication)- הוספת שכבת אבטחה נוספת עבור כניסה למערכת באמצעות קוד חד פעמי.

10. תיאור טכנולוגיה הנדסית

שפות תכנות:

Java 21

Java 21 היא שפת תכנות עילית מונחית עצמים (Object-Oriented Programming), המתאימה לפיתוח מערכות תוכנה מורכבות. השפה מאפשרת מימוש של רכיבי צד שרת ולוגיקה יישומית, וכן שילוב עם מסגרות עבודה המאפשרות פיתוח מערכת מלאה מקצה לקצה (Full-Stack).

מסגרת עבודה(Frameworks):

Spring Boot 3.5.9

Spring Boot 3.5.8 היא מסגרת עבודה לפיתוח יישומי צד שרת מבוססי Java, המספקת תצורה אוטומטית ותשתית מוכנה לפיתוח שירותי Web ו-API. המסגרת מאפשרת מימוש רכיבי Backend כגון שרת HTTP מובנה, לוגיקה יישומית בצד השרת, שכבת גישה לנתונים ובקרים לטיפול בבקשות HTTP. השימוש ב-Spring Boot מפשט את תהליך הפיתוח ומאפשר בנייה מסודרת של שכבת הלוגיקה והתקשורת במערכת.

Vaadin 24.9.9

Vaadin היא מסגרת עבודה לפיתוח ממשקי Web מבוססי Java בצד השרת, המאפשרת יצירת ממשק משתמש אינטראקטיבי ללא כתיבה ישירה של קוד Frontend. הסנכרון בין השרת לדפדפן מתבצע באופן אוטומטי באמצעות WebSocket ו-Push.

סביבת פיתוח:

VsCode 1.108

Visual Studio Code (VS Code) היא סביבת פיתוח קלה וגמישה, התומכת בפיתוח Java באמצעות הרחבות ייעודיות. הסביבה מאפשרת כתיבת קוד, ניפוי שגיאות, ניהול פרויקטים ועבודה עם מערכות ניהול גרסאות בצורה נוחה ויעילה.

במהלך הפיתוח נעשה שימוש בהרחבות וכלי עזר, כגון PlantUML ליצירת תרשימי ארכיטקטורה ותרשימי רצף, וכן הרחבות לפיתוח Java, Spring ו-MongoDB.

בפרויקט נעשה שימוש במספר ספריות מרכזיות, בהן Spring Web לפיתוח שירותי Web, Vaadin לפיתוח ממשק משתמש מבוסס Web, Spring Data MongoDB לניהול הנתונים ו-Spring Boot לניהול הנתונים ו-DevTools לצורכי פיתוח וטעינה מחדש.

11. מסד נתונים

המערכת עושה שימוש במסד נתונים מסוג NoSQL המאוחסן בסביבת ענן (כגון: MongoDB, Firebase) הבחירה במסד נתונים מסוג זה נובעת מהצורך בגמישות מבנה הנתונים, יכולת הרחבה (Scalability) והתאמה למערכות מודרניות מבוססות רשת. מסד נתונים NoSQL הוא מסד נתונים שאינו מבוסס על מבנה טבלאי קשיח ואינו מחייב סכימה קבועה מראש. מסדים אלו מאפשרים אחסון מידע במבנים דינמיים כגון מסמכים, אובייקטים או מפתחות-ערכים, ומתאימים במיוחד למערכות שבהן סוגי הנתונים, שדות המידע או נפח הנתונים עשויים להשתנות עם הזמן. גישה זו מקלה על פיתוח ותחזוקה של מערכות מורכבות, ומאפשרת התאמה מהירה לדרישות חדשות ללא שינוי מבני משמעותי במסד הנתונים.

מסד נתונים בענן הוא מסד נתונים המאוחסן ומנוהל על גבי תשתית מרוחקת, הנגישה דרך רשת האינטרנט. שימוש במסד נתונים בענן מאפשר זמינות גבוהה, שרידות נתונים, גיבויים אוטומטיים וניהול נוח של ההרשאות והמשאבים, ללא צורך בתחזוקת שרתים פיזיים בצד המפתח. בנוסף, פתרונות ענן מאפשרים הגדלה או הקטנה של משאבי האחסון והביצועים בהתאם לצורכי המערכת.

אוספים העיקריים במערכת:

• משתמשים (Users)

אוסף זה משמש לאחסון מידע על משתמשי המערכת.

שדות עיקריים:

- מזהה משתמש ייחודי
- שם משתמש

- סיסמה מוצפנת
- תאריך יצירת חשבון

מידע זה מאפשר ניהול משתמשים, אימות זהות ובקרת גישה למערכת.

• קבצים (Files)

אוסף זה משמש לאחסון מידע נלווה על הקבצים המאוחסנים בשרת.

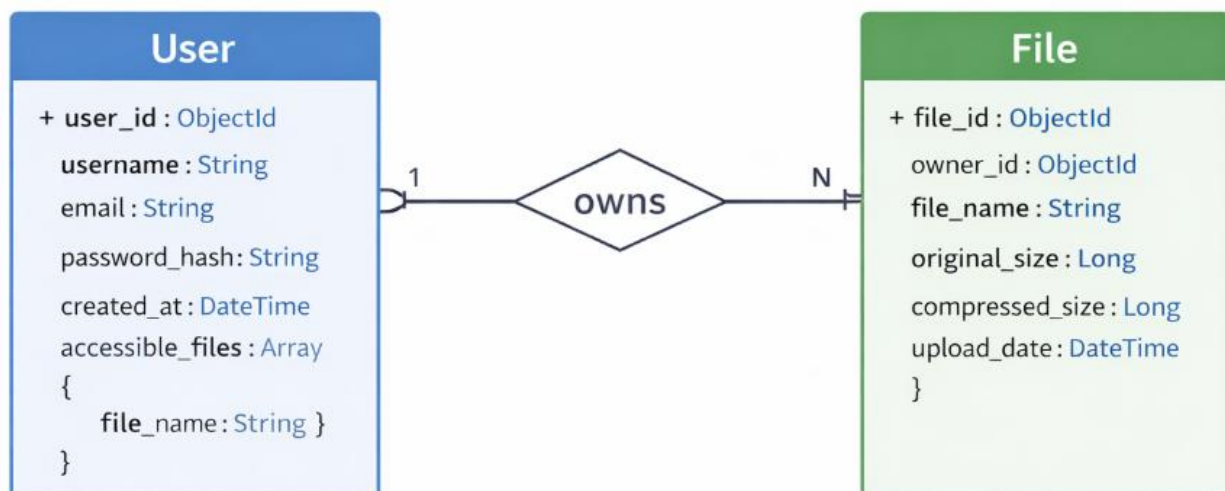
שדות עיקריים:

- מזהה קובץ
- מזהה המשתמש בעל הקובץ
- שם הקובץ
- גודל הקובץ (לפני ואחרי דחיסה)
- תאריך העלאה

הקבצים עצמם מאוחסנים במערכת האחסון של השרת, בעוד שמסד הנתונים שומר את המידע

הדרוש לניהול ושליפה.

להלן תרשים ERD:



מסד הנתונים משמש כרכיב ניהולי התומך בפעולת המערכת. הוא מאפשר זיהוי משתמשים, שיוך קבצים לבעליהם, ניהול הרשאות ושליפת מידע בצורה יעילה. הפרדה זו בין אחסון הקבצים לבין אחסון המידע הנלווה תורמת לסדר הנדסי, תחזוקה נוחה והרחבה עתידית של המערכת.

הקשר בין הישויות הוא קשר של אחד-לרבים (One-to-Many), שבו משתמש אחד יכול להיות משויך למספר קבצים, בעוד שכל קובץ משויך למשתמש אחד בלבד.

12. פרטים פורמליים**לוח זמנים:**

לסיים עד תאריך	שלבי עבודה
1.12.2025	בחירת פרויקט, חקירה ולמידה לעומק של נושאי הפרויקט
11.12.2025	כתיבה והגשת הצעת הפרויקט לאישור משרד החינוך
13.1.2026	מימוש הקוד של האלגוריתם המרכזי, ביצוע בדיקות ושיפורים
3.2.2026	בניית צד שרת
17.2.2026	בניית מסד הנתונים ושילובו
3.3.2026	בניית צד לקוח
24.3.2026	כתיבת ספר הפרויקט
7.4.2026	הגשת הפרויקט כולו (ספר + קוד) להגנה וקבלת ציון מגן

מנחה הפרויקט:

חתימת הסטודנט:



חתימת רכז המגמה: